1 Introduction

Today, in lecture 3, we will be covering data compression. We will be thinking about entropy as the number of bits required to describe the value of a random variable.

The main message of today's talk is that morally, the entropy of a random variable is the number of bits needed to describe it. We will see two different ways in which this is true.

2 Huffman Coding (a prefix code)

Say X is a discrete random variable (finite number of states). (Both Alice and Bob know the distribution of X).

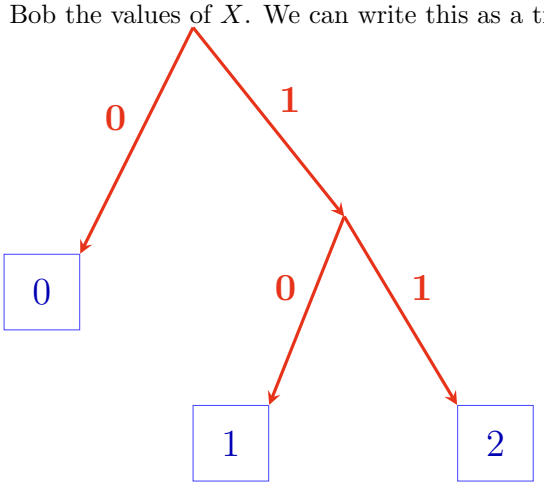
Theorem: Alice can encode X using less than $H(X)+1$ bits in expectation. From this encoding, Bob can retrieve X with probability 1.

2.1 Example (Back to Lecture 1)

x	$P(x)$	code
0	1/2	0
1	1/4	10
2	1/4	11

Table 1: Probability Distribution Table

We can use on bit to tell whether $X = 0$ or $X \in \{1, 2\}$. then, if $X \in \{1, 2\}$, another bit to tell whether $X = 1$ or $X = 2$. In expectation, it take 1.5 bits to tell Bob the values of X. We can write this as a tree.



See picture.

2.2 Prefix code

X gets values in $\{1, \ldots, m\}$ with probabilities $(p_1, \ldots, p_m)$, we want to place the symbols $1, \ldots, m$ on the leaves of a binary tree (a tree with out-degree 0 or 2). Each symbol is coded by the bits on the path that leads to it. (Prefix code is where no codeword is a prefix of a different codeword).

See picture for example.

The average code length for a prefix code is given by

$$L(T) = \sum_{i=1}^{m} p_i l_i,$$

where l_i is the length of path leading to symbol i (number of bits used to code i).

We want the optimal code, the code that minimizes

$$\sum_{i=1}^{m} p_i l_i$$

Using Huffman code we can get to average code length

$$H(p_1, \ldots, p_m) \leq \sum_{i=1}^{m} p_i l_i < H(p_1, \ldots, p_m) + 1$$

and if $p_1, \ldots, p_,$ are powers of 2, the average code length is exactly $H(p_1, \ldots, p_m)$.

2.3 The optimal code

Suppose that a tree T is optimal for symbols $\{1, \ldots, m\}$ with probabilities $(p_1, \ldots, p_m)$ (without loss of generality, we can assume $p_1 \geq p_2 \geq \cdots \geq p_m$).

Let v do be the deepest non-leaf vertex in T. Since T is optimal, w.l.o.g the symbols on the children of v are $m-1, m$ (as otherwise we can change them to be $m-1, m$ without increasing $\sum_{i=1}^{m} p_i l_i$)

See the picture.

Let T' be the tree generated from T by removing the children of v and placing the new symbol $* = \{m-1, m\}$ on v. T' is a tree (or code) for symbols $\{1, \ldots, m-2, *\}$, with probabilities $(p_1, \ldots, p_{m-2}, p_{m-1} + p_m)$. We have

$$L(T) = L(T'1) + (p_{m-1} + p_m) \cdot 1$$

Thus, T is optimal for $\{1, \ldots, m\}$ with probabilities (p_1) missed the rest

2.4 Huffman's algorithm

Input: $(p_1, \ldots, p_m)$ and sort them in descending order.

Find, by recursion, the best tree for probabilities $(p_1, \ldots, p_{m-2}, p_{m-1} + p_m)$ see the picture for the rest

2.4.1 Example

See the picture

2.5 $L(T)$ when $p_1, \ldots, p_m$ are all powers of 2

Lemma: If for every i, $p_i = 2^{-d_i}$, then in Huffman's tree T: $l_i = d_i$.

See the photo for the example.

2.5.1 First Proof

Assume, w.l.o.g. $d_1 \leq d_2 \leq \cdots \leq d_m$. Then $d_m = d_{m-1}$, as otherwise,

$$2^{d_m} = 2^{d_m} \cdot \sum_i p_i = \sum_i 2^{d_m} \cdot p_i = \sum_i 2^{d_m} \cdot 2^{-d_i}$$

has even left-hand side and odd right-hand side.

Maybe more explicitly, we could write

$$\sum_i 2^{d_m} \cdot 2^{-d_i} = \sum_{i=1}^{m} 2^{d_m - d_i} = 1 + \sum_{i=1}^{m-1} 2^{d_m - d_i}$$

where the right hand side would be odd if $d_{m-1} \neq d_m$.

2.5.2 Second Proof

Assume w.l.o.g. $d_1 \leq d_2 \leq \cdots \leq d_m$. Then $d_m = d_{m-1}$.

Thus $(p_1, \ldots, p_{m-2}, p_{m+1} + p_m)$ are all powers of 2 and by induction the claim is true for T' and hence also for T when we replace the leaf that corresponds to $\{m-1, m\}$ by a vertex with two leaves.

Hence

$$\sum_{i=1}^{m} p_i l_i = \sum_{i=1}^{m} p_i d_i = \sum_{i=1}^{m} p_i \log \frac{1}{p_i} = H(p_1, \ldots, p_m) = H(X)$$

2.6 For every T, we have $H(X) \leq L(T)$

Claim: For any binary tree T, with leaves $1, \ldots, m$

$$\sum_{i=1}^{m} 2^{-l_i} = 1$$

where l_i is the length of path to leaf i and the sum is over all leaves.

See picture for example.

2.6.1 Proof

Consider a random walk from the root to the leaves that at each step goes from a non-leaf vertex v to each of its two children with probability $1/2$ each, until reaching a leaf. Then the probability to reach leaf i is 2^{-l_i} and since the random walk reaches exactly one leaf the total probability must be 1 (alternatively, a proof by induction).

2.7 For every T, we have $H(X) \leq L(T)$

Proof: For any binary tree T, with leaves $1, \ldots, m$ we now know $\sum_{i=1}^{m} 2^{-l_i} = 1$, where l_i is the length of path to leaf i.

Let $q = \left(2^{-l_1}, \ldots, 2^{-l_m}\right)$. thus q is a distribution over $\{1, \ldots, m\}$. We have

$$\begin{aligned} 0 \leq \mathrm{D}\left(p|q\right) &= \sum_i p_i \log \frac{p_i}{q_i} \\ &= \sum_i p_i \log p_i + \sum_i p_i \log \left(\frac{1}{q_i}\right) \\ &= -H(X) + \sum_i p_i l_i \end{aligned}$$

Thus

$$H(X) \leq L(T)$$

2.8 $L(T) < H(X) + 1$

We know the lower bound, but now we can find the upper bound (on an optimal code, Huffman's code).

First step of the proof, for every i, let $q_i = 2^{-d_i}$ be the largest power of 2 that is less than or equal to p_i. Thus, $\frac{p_i}{2} < q_i \leq p_i$. Note that $\sum_i q_i \leq 1$.

This means that $(q_1, \ldots, q_m)$ might not be a distribution, but we can fix this by extending.

Extend $q_1, \ldots, q_m$ to a distribution over $m' \geq m$ elements by adding $q_{m+1}, \ldots, q_{m'}$ that are powers of 2. (We won't prove it, but it's not hard.)

Let T be Huffman's tree for $q_1, \ldots, q_{m'}$. We use T as a tree for $p_1, \ldots, p_m$ (by giving probability 0 to symbols in $\{m+1, \ldots, m'\}$). Now we can write

$$\begin{aligned} L(T) &= \sum_{i=1}^{m} p_i l_i = \sum_{i=1}^{m} p_i d_i \\ < \sum_{i=1}^{m} p_i \log \frac{1}{p_i/2} &= \sum_{i=1}^{m} p_i \left(\log \frac{1}{p_i} + 1\right) = H(X) + 1 \end{aligned}$$

Since Huffman's tree is optimal, this must be true also for Huffman's.

2.8.1 Remark: what the proof of $L(T) < H(X) + 1$ is really doing

There are two distributions in the proof: p, the true probabilities of the symbols, and q, the dyadic distribution we build from p. The tree is built for q but scored with p.

Step 1: build some good code T for p. Since every $q_i = 2^{-d_i}$ is a power of 2, the Lemma ("$L(T)$ when $p_1, \ldots, p_m$ are all powers of 2") applies to q: in Huffman's tree T for q, we get $l_i = d_i$. This is why $\sum_i p_i l_i = \sum_i p_i d_i$ holds even though p need not be dyadic. The lengths come from q, and p only supplies the weights. Then $q_i > p_i/2$ gives

$$d_i = \log \frac{1}{q_i} < \log \frac{2}{p_i} = \log \frac{1}{p_i} + 1,$$

and so $L_p(T) = \sum_i p_i d_i < H(X) + 1$.

(Huffman for q is just a convenient way to get a prefix tree with lengths exactly d_i. Any such tree works. One could instead use Kraft's inequality directly, since $\sum_i 2^{-d_i} \leq 1$. That gives the Shannon code.)

Step 2: use optimality. Let T^* be Huffman's tree for p. Since T^* is optimal for p, it does at least as well as any prefix code for p, T included:

$$L_p(T^*) \leq L_p(T) < H(X) + 1.$$

Caveats.

- Both expected lengths are computed under p: $L_p(T) = \sum_i p_i l_i$, *not* $\sum_i q_i l_i$.
- T need not be suboptimal. All we need is that it is *some* valid prefix code for p.
- The argument does *not* show that every prefix code has $L < H(X) + 1$. That is false: a bad code can be arbitrarily long, e.g. by giving a long codeword to a very likely symbol. The bound holds for the optimal code because it is optimal.

2.9 Back to Huffman's code (a prefix code)

X: a discrete random variabel (finite number of states)

Theorem: Alice can encode X using less than $H(X) + 1$ bits *in expectation*. From this encoding, Bob can retrieve X with probability 1.

Corollary: By Markov's inequality, Alice can encode X using $\mathcal{O}(H(X))$ bits, in the worst case. From this encoding, Bob can retrieve X with probability $1 - \epsilon$.

3 Review: Approximating Multinomial Coefficients

Let $(p_1, \ldots, p_m)$ be a distribution and thus $\sum_{i=1}^{m} p_i = 1$. Let $n \in \mathbb{N}$ and let $(n_1, \ldots, n_m)$ be such that $n_i = np_i$ and thus $\sum_{i=1}^{m} n_i = n$. Consider the multinomial coefficient

$$\frac{n!}{n_1! \cdot n_2! \cdot \ldots \cdot n_m!} = \frac{n!}{(np_1)! \cdot (np_2)! \cdot \ldots \cdot (np_m)!}$$

As $n \to \infty$, we can write

$$\frac{1}{n} \cdot \log(\cdots)$$

see the picture.

4 Data Compression

4.1 Example

Let $X_1, \ldots, X_n$ be independent random variables, such that, each X_i is 0 with probability p and 1 with probability $1 - p$.

With high probability, about pn of them are 0. Assume that exactly $k = pn$ of them are 0 and $(1-p)\,n$ are 1. Then, the number of possibilities for $X_1, \ldots, X_n$ is $\binom{n}{k} \approx 2^{n \cdot H(p,1-p)}$.

Thus, around $n \cdot H(p, 1-p)$ bits are needed to tell in which possibility we are, and hence to describe the values of $X_1, \ldots, X_n$

$$n \cdot H(p, 1-p) = n \cdot H(X_i) = H(X_1, \ldots, H_n)$$

4.1.1 A Protocol for Describing $X_1, \ldots, X_n$

If $X_1, \ldots, X_n$ are i.i.d and X_i has the distribution $(p, 1-p)$:

1. Send the number of zeros: $k = n - \sum X_i \approx pn$ (with high probability): $\log n$ bits (negligible)
2. Send the exact values of $X_1, \ldots, X_n$, among all possibilities with k zeros: $\approx n \cdot H(p, 1-p)$ bits

4.2 More generally

If $X_1, \ldots, X_n$ are i.i.d and X_i has the distribution $(p_1, \ldots, p_m)$, where $n >> m$ $(n \to \infty)$, we can describe $X_1, \ldots, X_n$ but sending about $n \cdot H(p_1, \ldots, p_m)$ bits:

1. Send the empirical distribution $(n_1, \ldots, n_m)$, where $n_i \approx p_i n$ is the number of X_j-s that have the value i: $m \log n$ bits (negligible)
2. Send the exact values of $X_1, \ldots, X_n$ among all possibilities with that empirical distribution: $\approx n \cdot H(p_1, \ldots, p_m)$ bits

4.3 Shannon's Source Coding Theorem

If $X_1, \ldots, X_n$ are independent and identically distributed, Alice can encode $X_1, \ldots, X_n$ using $H(X_i) \cdot n + \wr(n)$ bits (in the worst case). From this encoding, Bob can retrieve $X_1, \ldots, X_n$ with probability $1 - \wr(1)$.

(Both Alice and Bob know the distribution of $X_1, \ldots, X_n$)

We will give a more formal general proof that proves even more than that (using the Asymptotic Equipartition Theorem).

Similar theorems, that roughly $H(X_1, \ldots, X_n)$ bits are needed to describe all variables $X_1, \ldots, X_n$, are also true in more general settings (such as Markov chains and other stochastic processes).

5 Asymptotic Equipartition Theorem

5.1 Law of Large Numbers

If $Z_1, \ldots, Z_n$ are i.i.d, then

$$\frac{Z_1 + \cdots + Z_n}{n} \to \mathbb{E}(Z_i)$$

(convergence in distribution equals convergence of the cumulative distribution function). That is, for every $\epsilon > 0$,

$$P_{Z_1,\ldots,Z_n}\left[|\frac{Z_1 + \cdots + Z_n}{n} - \mathbb{E}(Z_i)| > \epsilon\right] \to 0$$

5.2 Asymptotic Equipartition Theorem: Informal

Let $X_1, \ldots, X_n$ be i.i.d. Consider the random variable $-\log P(X_1, \ldots, X_n)$. Note that $\mathbb{E}(-\log P(X_1, \ldots, X_n)) = H(X_1, \ldots, X_n)$. We will show that almost always $-\log P(X_1, \ldots, X_n)$ is very close to its expectation.

5.3 Asymptotic Equipartition Theorem: Formal

If $X_1, \ldots, X_n$ are i.i.d, then

$$\frac{-\log P(X_1, \ldots, X_n)}{n} \to H(X_i)$$

(convergence in distribution equals convergence of the cumulative distribution function). That is, for every $\epsilon > 0$,

$$P_{Z_1,\ldots,Z_n}\left[|\frac{-\log P(X_1, \ldots, X_n)}{n} - H(X_i)| > \epsilon\right] \to 0$$

5.3.1 Proof

Since $X_1, \ldots, X_n$ are independent,

$$P(X_1, \ldots, X_n) = P(X_1) \cdot \ldots \cdot P(X_n)$$

Hence by the law of large numbers

$$\frac{-\log P(X_1, \ldots, X_n)}{n} = \frac{-\sum_i \log P(X_i)}{n} \to \mathbb{E}\left(-\log P(X_i)\right) = H(X_i)$$

5.4 Additional Form: I

If $X_1, \ldots, X_n$ are i.i.d, then

$$P_{Z_1,\ldots,Z_n}\left[\left|\frac{-\log P(X_1, \ldots, X_n)}{n} - H(X_i)\right| > \epsilon\right] \to 0$$

which implies

$$P\left[H(X_i) - \epsilon \leq \frac{-\log P(X_1, \ldots, X_n)}{n} \leq H(X_i) + \epsilon\right] \to 1$$

$$P\left[2^{H(X_i)-\epsilon} \leq \frac{-\log P(X_1, \ldots, X_n)}{n} \leq H(X_i) + \epsilon\right] \to 1$$

See the picture.

Thus, $P(X_1, \ldots, X_n)$ is with high probability close to $2^{-H(X_1,\ldots,X_n)}$

5.5 Additional Form: II

Let $X_1, \ldots, X_n$ be i.i.d. Let $X = (X_1, \ldots, X_n)$. For every n and $\epsilon > 0$, denote by $A_{n,\epsilon}$ the set of "typical sequences":

$$A_{n,\epsilon} = \left\{a = (a_1, \ldots, a_n) \,\middle|\, 2^{-n\cdot(H(X_i)+\epsilon)} \leq P(X = a) \leq 2^{-n\cdot(H(X_i)-\epsilon)}\right\}$$

See the picture.

6 Data Compression

We think of $A_{n,\epsilon}$ as the set of states that $(X_1, \ldots, X_n)$ can take (as the rest is negligible and will result in negligible error). We have

$$|A_{n,\epsilon}| \approx 2^{H(X_1,\ldots,X_n)}$$

Alice can describe $(X_1, \ldots, X_n)$ by sending about $H(X_1, \ldots, X_n)$ bits. From this description, Bob can recover $(X_1, \ldots, X_n)$ exactly, with high probability.

This is called Shannon's Source Coding Theorem. There are similar theorems for non i.i.d conditions.

6.1 Lower Bound

Is Shannon's source coding theorem tight?

Could Alice send significantly less than $H(X_1, \ldots, X_n)$ bits from which Bob ... lost the rest

6.1.1 Proof

For simplicity, assume that $X_1, \ldots, X_n$ are bits. Assume that Alice encodes $X = (X_1, \ldots, X_n)$ by applying a function f and obtains $Y = f(x)$. Assume that Y is of length m bits. Assume that from Y, Bob recovers X with high probability by applying a function g, obtaining $g(Y)$

$$X \to f(X) \to g(f(X))$$

Assume that $P(X = g(Y)) \geq 1 - \epsilon$.

By Question 9 in Problem Set 1[1],

$$H(X|Y) \leq H(\epsilon, 1-\epsilon) + \epsilon n < \epsilon n + 1$$

We hence have,

$$I(X;Y) = H(Y) - H(Y|X) \leq H(Y) \leq m$$
$$I(X;Y) = H(X) - H(X|Y) > H(X) - \epsilon n - 1$$

Thus,

$$m > H(X) - \epsilon n - 1$$

This is an example of the power of the equalities and inequalities of information theory

[1] There was a hint of constructing an indicator variable.